JSON preview benchmark

Updater prototype compared with clean master | JDK 28-internal | measured 15 Oct 2026

THE RESULT

Fresh scalar creation plus first access is faster and allocates less, but repeated reads of initialized scalars are slower. First JSON use also takes longer.

FRESH SCALAR ACCESS

+58 to +63%

higher throughput

INITIALIZED READ

-51 to -54%

lower throughput

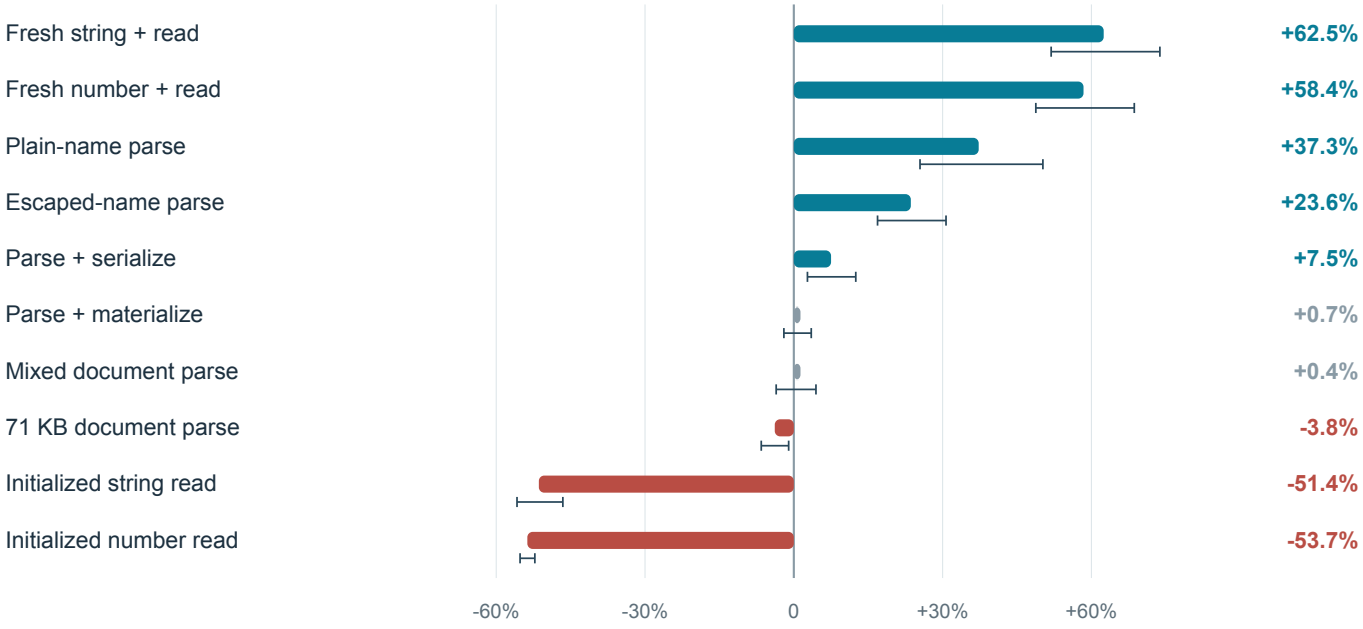
FIRST JSON PARSE

+5.7 to +6.7 ms

median paired delay

Throughput change after warmup

Higher is better. Bars show paired branch/master change; whiskers show exploratory 95% intervals.



A positive change means more operations per second on the prototype branch.

WHAT THE CHART SHOWS

Fresh number and string creation plus first access improve 58.4% and 62.5%. Already initialized asInt() and asString() reads fall 53.7% and 51.4%. Small member-name parses improve; the 32-row mixed parse is effectively unchanged.

The existing 71 KB parse benchmark falls 3.8%, despite lower allocation. The next page gives exact throughput, intervals, and allocated bytes per operation.

Throughput and allocation

JMH 1.37 | one thread | four paired forks per workload | higher ops/s is better

PARSING WORKLOADS

Workload	Master	Branch	Change / 95% range	B/op M / B
Existing 71 KB parse	16,297	15,669	-3.8% -6.6 to -1.0%	178,880 / 165,232
Mixed parse, 32 rows	202,027	202,827	+0.4% -3.6 to +4.5%	32,000 / 21,744
Parse, plain names	729,606	1,000,965	+37.3% +25.4 to +50.2%	9,696 / 5,584
Parse, escaped names	516,090	638,186	+23.6% +16.9 to +30.7%	10,072 / 5,960
Parse + materialize	94,969	95,634	+0.7% -2.0 to +3.5%	40,368 / 30,112
Parse + serialize	95,277	102,454	+7.5% +2.7 to +12.5%	45,968 / 35,712

SCALAR VALUE WORKLOADS

Workload	Master	Branch	Change / 95% range	B/op M / B
Create number + asInt()	17,001,996	26,943,572	+58.4% +48.8 to +68.6%	312 / 184
Create string + asString()	8,315,609	13,505,640	+62.5% +51.9 to +73.8%	420 / 356
Initialized number asInt()	750,233,285	347,233,957	-53.7% -55.2 to -52.2%	about 0 / 0
Initialized string asString()	744,519,332	361,814,626	-51.4% -55.8 to -46.6%	about 0 / 0

READING THE RESULT

Allocation falls across the measured allocating workloads. Mixed parsing uses 32,000 versus 21,744 B/op (-32%), while its throughput is roughly unchanged. The 71 KB parse allocates 7.6% less but is 3.8% slower.

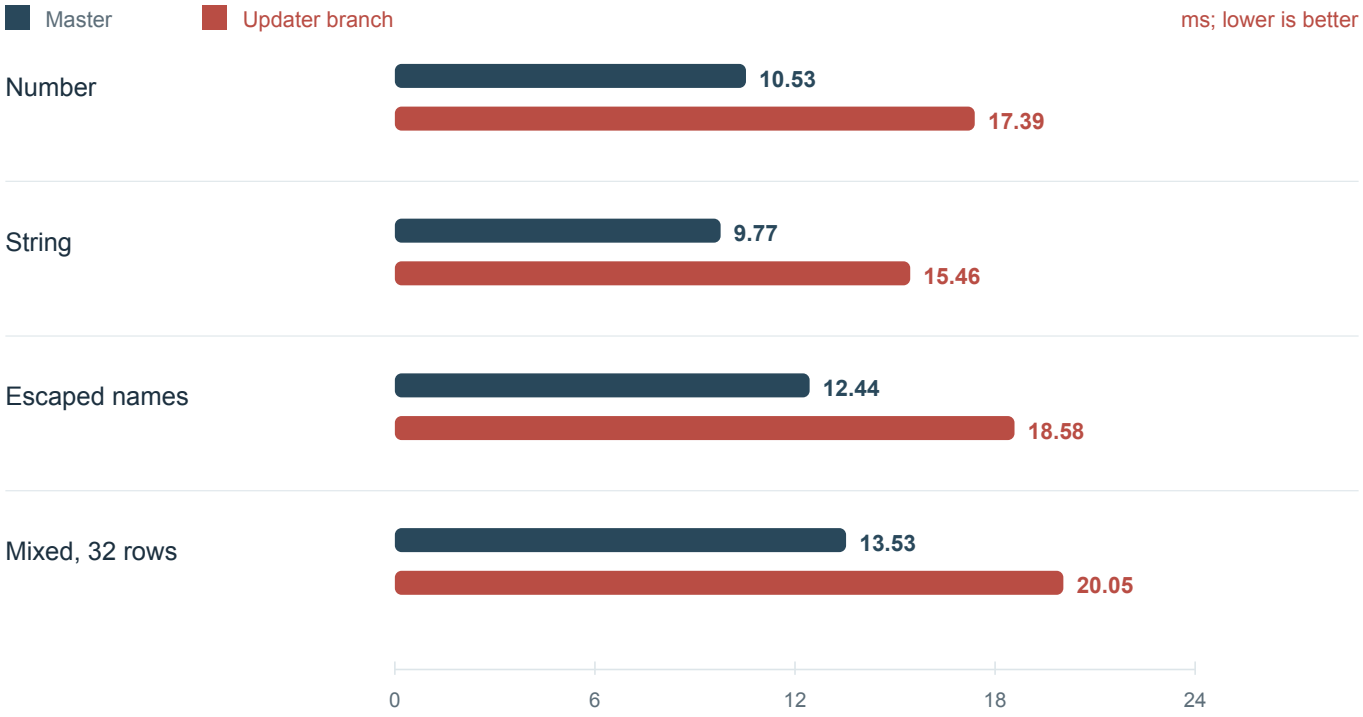
The initialized-read loss is consistent with updater and method-handle dispatch; the allocation savings are consistent with removing per-instance lazy holders. These are implementation-based inferences, not isolated causal estimates.

Change: geometric paired ratio (exploratory 95% t range); B/op: JMH gc.alloc.rate.norm.

First-use cost

30 alternating fresh-JVM pairs per mode | the branch is slower in every first-parse pair

Median first Json.parse time



Paired delay and process wall time

Mode	Paired +ms	Slower pairs	Wall master	Wall branch
Number	+6.73	30/30	62.00 ms	68.47 ms
String	+5.68	30/30	60.44 ms	66.36 ms
Escaped names	+6.10	30/30	67.75 ms	72.61 ms
Mixed, 32 rows	+6.58	30/30	70.88 ms	77.72 ms

METHOD AND LIMITS

JMH 1.37, one thread: four paired forks per workload, with three 1-second warmup and five 1-second measurement iterations. -prof gc measured B/op. Both release/server macOS arm64 JDK 28 images used the same benchmark classes and inputs.

Cold test: first-parse timing excludes JVM launch; process wall includes launch and module resolution. Each mode used 30 alternating JVM pairs; outputs were validated.

Scope: one machine and four forks provide directional evidence. The branch changes more than JSON, so this full-image comparison cannot isolate the updater edits.

Master: 00f4e1dfe37a. Raw results and harness: build/json-benchmark-2026-10-15/